K.R. MANGALAM UNIVERSITY

School of Engineering and Technology

Lab Assignment – 4:

Backtracking, Branch & Bound, and String Matching

Submitted by:
Satyam Singh
(2401420022)

Submitted to:
Dr. Aarti

Assignment Tasks

Task 1: Backtracking — Airline Crew Scheduling

Overview

This task solves a crew scheduling problem using backtracking. Given a list of flights (each with a start and end time) and a set of crew members, the goal is to assign each flight to a crew member such that no crew member is assigned two overlapping flights.

Real-World Use

Airlines must assign pilots and cabin crew to flights while respecting rest times and avoiding schedule conflicts. This is a classic Constraint Satisfaction Problem (CSP) where backtracking explores all valid assignments and abandons a branch the moment a conflict is detected.

Problem Setup

Flights: F1 (09:00–11:00), F2 (10:00–12:00), F3 (12:00–14:00)

Crew Members: C1, C2

Constraint: No crew member may be assigned two flights whose time windows overlap.

Algorithm

1. Try assigning each crew member to the current flight.
2. Check if the assignment is valid (no time overlap with existing assignments).
3. If valid, record the assignment and recurse to the next flight.
4. If recursion succeeds, return True. If not, backtrack (remove assignment) and try next crew.
5. If no crew member is valid, return False — backtrack further up.

Complexity:

Time: O(k^n) in the worst case — k crew members, n flights. Pruning reduces practical runtime significantly.

Space: O(n) — recursion depth equals number of flights.

Overlap Check Logic

```
def is_valid(flight, crew_member):
    for f, c in assignment.items():
```

```
        if c == crew_member:
            if not (flight[2] <= f[1] or flight[1] >= f[2]):
                return False
    return True
```

Backtracking Recursive Function

```
def backtrack(i):
    if i == len(flights):          # All flights assigned
        return True
    for c in crew:
        if is_valid(flights[i], c):
            assignment[flights[i]] = c
            if backtrack(i + 1):
                return True
            del assignment[flights[i]]  # Backtrack
    return False
```

Observation

F1 and F2 overlap (09–11 and 10–12 share time), so they must be assigned to different crew members. F3 (12–14) does not overlap with F1 (ends at 11) so the same crew member can handle both. A valid assignment is: F1→C1, F2→C2, F3→C1. Backtracking finds this automatically by exploring and pruning the search tree.

Result / Output

```
{('F1', 9, 11): 'C1', ('F2', 10, 12): 'C2', ('F3', 12, 14): 'C1'}
```

Task 2: Branch and Bound — Minimise Crew Usage

Overview

Branch and Bound extends backtracking by adding an optimality criterion: here, we minimise the total number of distinct crew members used. The algorithm prunes any branch whose current crew count already meets or exceeds the best solution found so far.

Difference from Backtracking

Property	Backtracking	Branch and Bound
Goal	Find any feasible solution	Find the optimal (minimum cost) solution
Pruning Criterion	Constraint violation only	Constraint violation + cost bound
Completeness	Stops at first valid solution	Explores all promising branches
Output	One valid assignment	Best assignment with minimum crew

Bounding Function

At each recursive step, we check: if the number of crew members already used in the current partial assignment equals or exceeds the best known complete solution, we prune (return early). This avoids exploring branches that cannot improve upon the current best.

```
def branch_and_bound(i, current):
    global best, best_assign
    if i == len(flights):               # Complete assignment
        cost = count_used(current)
        if cost < best:
            best = cost
            best_assign = current.copy()
        return
    if count_used(current) >= best:     # PRUNE: can't improve
        return
    for c in crew:
        valid = True
        for f, cc in current.items():
            if cc == c:
                if not (flights[i][2] <= f[1] or flights[i][1] >= f[2]):
                    valid = False; break
        if valid:
            current[flights[i]] = c
            branch_and_bound(i + 1, current)
            del current[flights[i]]
```

Observation

The Branch and Bound solution finds the assignment that uses the fewest crew members. For the given flights, the optimal solution uses only 2 crew members (C1 and C2), with F1→C1, F2→C2, F3→C1. The pruning condition prevents exploring any branch that already uses 2 or more crew when a 2-crew solution already exists — drastically reducing the number of recursive calls compared to exhaustive backtracking.

Result / Output

```
Assignment 4 - Task 2 (Branch and Bound: Minimize Crew Usage)
{('F1', 9, 11): 'C1', ('F2', 10, 12): 'C2', ('F3', 12, 14): 'C1'}
```

Task 3: String Matching — Naive Algorithm

Overview

The Naive string matching algorithm searches for all occurrences of a pattern within a text by sliding the pattern one position at a time and checking for a character-by-character match at each position. It also counts the total number of character comparisons made.

Real-World Use

String matching underpins grep (Unix search), browser Ctrl+F, antivirus signature scanning, plagiarism detection, DNA subsequence search in bioinformatics, and SQL LIKE queries.

Algorithm

1. For each position i in text (0 to n−m), align the pattern at position i.
2. Compare text[i+j] with pattern[j] for j = 0 to m−1.
3. If all m characters match, record a match at index i.
4. Increment comparison counter at every character check.

Complexity:

Time: O(n × m) worst case (e.g., text = AAAA...A, pattern = AAAB)

Space: O(1) — no auxiliary data structures beyond loop variables.

Code

```
def naive_search(text, pattern):
    n, m  = len(text), len(pattern)
    count = 0
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            count += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        if match:
            print('Pattern found at index', i)
    print('Comparisons:', count)
```

Test Input

Text: "ABABDABACDABABCABAB"

Pattern: "ABABCABAB"

Observation

The pattern 'ABABCABAB' is found at index 10 in the text. The naive approach performs significantly more comparisons than necessary because it restarts from scratch at each position even when partial matches contain reusable information. This inefficiency is addressed by algorithms like KMP which reuse prefix information.

Result / Output

```
Pattern found at index 10
Comparisons: 29
```

Task 4: Experimental Comparison — Naive Search at Scale

Overview

This task measures the execution time of the naive string matching algorithm for increasing text sizes (100, 500, 1000 characters) to empirically confirm its O(n × m) growth. A worst-case input is used: text = 'A' × size + 'B', pattern = 'AB'.

Why Worst Case?

When text = 'AAA...AB' and pattern = 'AB', at every position the algorithm matches the first character 'A' before failing on 'B'. This forces the maximum number of comparisons (~n × m), making the scaling behaviour clearly visible.

Timing Code

```
import time

def test(size):
    text    = 'A' * size + 'B'
    pattern = 'AB'
    start   = time.time()
    naive_search(text, pattern)
    end     = time.time()
    return end - start

for size in [100, 500, 1000]:
    print(size, test(size))
```

Expected Results Table

Text Size (n)	Pattern Length (m)	Max Comparisons (n×m)	Expected Trend
100	2	200	Baseline
500	2	1000	~5× slower
1000	2	2000	~10× slower

Algorithm Comparison

Algorithm	Time Complexity	Space	Handles Worst Case Well?
Naive	O(n × m)	O(1)	No — very slow

KMP	O(n + m)	O(m)	Yes — linear always
Rabin-Karp	O(n + m) avg	O(1)	Yes — hash-based
Boyer-Moore	O(n/m) best	O(σ)	Yes — fast in practice

Observation

As text size doubles, execution time approximately doubles — confirming linear growth in n for a fixed short pattern. On larger text (n = 10,000+) with long patterns, the quadratic nature would become much more pronounced. KMP and Boyer-Moore would remain fast regardless, making them far preferable for production string search use cases.

Result / Output

```
Pattern found at index 99
Comparisons: 200
Text size 100: 0.000200 seconds
Pattern found at index 499
Comparisons: 1000
Text size 500: 0.000289 seconds
Pattern found at index 999
Comparisons: 2000
Text size 1000: 0.000341 seconds
```

Task 5: Conceptual Programming Tasks

5A — Why Backtracking for Crew Scheduling?

Crew scheduling is a Constraint Satisfaction Problem (CSP). Backtracking is the canonical approach for CSPs: it builds a solution incrementally and abandons any partial solution the moment it violates a constraint — avoiding the exponential cost of generating all complete assignments first and checking them after.

5B — Pruning in Branch and Bound

The key bounding condition is: if count_used(current) >= best, prune. This works because adding more flights can only keep or increase crew count — never decrease it. So any branch that already matches the best is guaranteed not to improve it, and can be safely discarded.

5C — Why Naive Search Has O(n × m) Worst Case

In the worst case (text = 'AAA...AB', pattern = 'AAB'), for every starting position i, the algorithm performs m−1 successful comparisons before failing. Since there are n−m+1 starting positions and m−1 comparisons each, total = (n−m+1)(m−1) ≈ O(n × m).

5D — NP-Hardness of Crew Scheduling

The general crew scheduling problem (minimise crew given unlimited flights and arbitrary constraints) is NP-hard because it reduces to graph colouring — a known NP-complete problem. Our backtracking solution is exponential O(k^n) in the worst case, which is why it fails to scale beyond ~10 flights.

5E — Extendability

Real airline scheduling adds: rest time between flights, crew qualifications per aircraft type, base airport constraints, cost per crew member, and union rules on maximum flying hours. These additional constraints would be encoded as extra checks in is_valid() and additional cost dimensions in the bounding function.

GitHub Repository: https://github.com/Code-trac/ada_lab